03 / GUÍA DEL EJERCICIO

EX29 — Rotaciones y matrices 2D

Class 18 · Parte matemática y aplicación física

Objetivo y alcance
Construir una matriz de rotación 2×2, utilizarla para girar vectores, verificar resultados algebraicos e interpretar una fuerza expresada en ejes distintos. Un solo archivo Python. No se requieren JSON, `pytest`, clases ni lectura de datos.

1. Ubicación del ejercicio

```
Python_Engineering/
  02_numerical_methods/
    algebra/
      class_18_matrices/
        EX28_matrix_operations.py
        EX29_rotation_2D.py       <-- NEW
        docs/
          ES/
          EN/
```

Archivo de trabajo: `EX29_rotation_2D.py`. Usar NumPy con `import numpy as np`.

2. Concepto que debe implementarse primero: $R(\theta)$

Antes de rotar ningún vector, definir el ángulo θ y construir su matriz de rotación general:

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

Esta es una expresión general, válida para cualquier ángulo. En Python no se escribe una matriz simbólica: se elige un valor numérico de θ, se calcula su seno y coseno y se monta la matriz numérica. Al cambiar el ángulo, hay que volver a calcular seno, coseno y matriz (por ahora sin crear una función propia).

1. Elegir `theta_deg`, por ejemplo 30°; convertirlo a `theta_rad`.
2. Calcular seno y coseno de `theta_rad`.
3. Construir R como array 2×2, con los signos y posiciones correctos.
4. Multiplicar Rv usando `@` y comprobar el resultado.
5. Cambiar `theta_deg` y reconstruir R para repetir el experimento.

Idea de aprendizaje: no se trata de almacenar varias matrices ya resueltas, sino de saber construir $R(\theta)$ para el ángulo que se necesite. Más adelante encapsularemos este procedimiento en una función reutilizable.

3. Hipótesis del modelo y notación

Todos los vectores son **columnas matemáticas** con dos componentes, representados como arrays NumPy de forma `(2,)`. El plano es cartesiano de mano derecha y los ángulos positivos son antihorarios. Se trabajará con vectores **sin unidades** en la Parte A y con fuerzas en Newtons en la parte B.

4. Parte A0 — Ejemplo algebraico resuelto (verificación)

Se considera $\mathbf{v} = (2, 1)^T$ y un giro antihorario de 90°. Realizar el cálculo manual:

$$R(90°) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \qquad R(90°)\mathbf{v} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}.$$

Reproducirlo con NumPy y comprobar que la norma inicial y final es $\sqrt{5} \approx 2.2361$. Este apartado sirve como control de la implementación, no como un problema independiente complejo.

5. Parte A1 — Giro algebraico de 30 grados

Datos: $\mathbf{a} = (4, 0)^T$ y $\theta = +30°$.

1. Convertir el ángulo a radianes.
2. Calcular seno y coseno y construir $R(30°)$.
3. Obtener $\mathbf{a}' = R(30°)\mathbf{a}$ con `@`.
4. Calcular $\|\mathbf{a}\|_2$ y $\|\mathbf{a}'\|_2$; comparar ambas normas.
5. Verificar matemáticamente al menos una componente sin usar Python.

Resultado de control: $\mathbf{a}' = (2\sqrt{3}, 2)^T \approx (3.4641, 2)^T$.

6. Parte A2 — Giro de 90 grados y comprobación inversa

Datos: $\mathbf{q} = (3, -2)^T$ y $\theta = +90°$.

1. Rotar el vector y comprobar que $\mathbf{q}' = (2, 3)^T$.
2. Aplicar la transpuesta $R(90°)^T$ al vector transformado. ¿Se recupera $\mathbf{q}$?
3. Utilizar `np.allclose()` para obtener un único valor lógico de comparación.

7. Parte A3 — Composición de dos rotaciones

Datos: $\mathbf{p} = (2, 0)^T$, primer giro $+30°$, segundo giro $-60°$ (ambos respecto a ejes fijos).

1. Aplicar ambas rotaciones una después de otra.
2. Comparar el resultado con una única rotación de $-30°$.
3. Comprobar la conservación de la norma y la equivalencia aproximada de los arrays.

Resultado de control: $\mathbf{p}_{\text{final}} = (\sqrt{3}, -1)^T \approx (1.7321, -1)^T$.

8. Parte B — Ingeniería: rotación de una fuerza

Una fuerza **física** de 10 N actúa inicialmente sobre el eje x positivo. Su dirección gira $+30°$ mientras los ejes globales permanecen fijos.

$$\mathbf{F}_{\text{inicial}} = \begin{pmatrix} 10 \\ 0 \end{pmatrix} \text{ N}.$$

Programar el vector de fuerza final, su norma y la comprobación de conservación de magnitud. Interpretar el signo de cada componente y escribir unidades en los resultados impresos. **Resultado de control:** $\mathbf{F}' = (5\sqrt{3}, 5)^T$ N $\approx (8.6603, 5)^T$ N; norma 10 N.

9. Parte C — Cambio de referencia (vector físico sin girar)

Una fuerza fija tiene componentes **globales** $\mathbf{F}_G = (3, 4)^T$ N. Ahora los **ejes** de referencia giran $+90°$ en sentido antihorario. La fuerza no se mueve.

1. Calcular las componentes $\mathbf{F}_L = R(90°)^T \mathbf{F}_G$ en los ejes girados.
2. Recuperar el vector global mediante $R(90°)\mathbf{F}_L$.
3. Comparar las normas y explicar en una frase qué ha cambiado y qué no.

Resultado de control: $\mathbf{F}_L = (4, -3)^T$ N y $\|\mathbf{F}_L\|_2 = 5$ N.

10. Lista de verificación

- Las dimensiones son coherentes: R de forma `(2,2)` y vector `(2,)`.
- No se confunden `*` y `@` ni grados y radianes.
- Se distinguen *rotación activa* $R\mathbf{v}$ y *cambio de base pasivo* $R^T\mathbf{v}$.
- Los resultados coinciden con las comprobaciones matemáticas, aceptando pequeños errores de coma flotante.

11. Ayudas de código: consultar al final

Estas instrucciones son **herramientas**, no la solución del ejercicio. El orden del algoritmo y la elección de variables los decide quien programa.

```python
import numpy as np

angle_rad = np.deg2rad(30.0)  # degrees -> radians
c = np.cos(angle_rad)         # scalar cosine
s = np.sin(angle_rad)         # scalar sine

# Matrix-vector product: R @ vector
# Inverse rotation for an orthogonal R: R.T @ rotated
# Approximate array comparison: np.allclose(result, expected)
```